

## TUGAS PRAKTIKUM PEKAN 9

### 1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (Breadth FirstSearch) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph.

### 2. Kode Program

```
1 package pekan9_2511532005;
2 import javax.swing.*;
3 import javax.swing.border.*;
4 import java.awt.*;
5 import java.util.*;
6 import java.util.List;
7
8 public class PetaKampus_2511532005 extends JFrame {
9     private List<String> vertices_2005;
10    private Map<String, List<String>> adjlist_2005;
11    private Map<String, Point> positions_2005;
12
13    private List<String> currentPath_2005;
14    private Set<String> currentVisited_2005;
15    private int currentExplored_2005;
16
17    private GraphPanel_2005 graphPanel_2005;
18    private JComboBox<String> comboStart_2005;
19    private JComboBox<String> comboGoal_2005;
20    private JButton btnBFS_2005;
21    private JButton btnDFS_2005;
22    private JButton btnReset_2005;
23    private JLabel labelJalur_2005;
24    private JLabel labelNodeDikunjungi_2005;
25    private JLabel labelJumlahNode_2005;
26
27    public PetaKampus_2511532005() {
28        setTitle("PENCAIRAN JALUR MENGGUNAKAN BFS DAN DFS - NIM 2511532005");
29        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
30        setSize(900, 700);
31        setLocationRelativeTo(null);
32        setLayout(new BorderLayout());
33
34        initializeGraph_2005();
35        setupGUI_2005();
36        displayGraph();
37    }
38
39    private void initializeGraph_2005() {
40        vertices_2005 = Arrays.asList(
41            "Rektorat", "Perpus", "FTI", "PKM",
42            "Lab Jaringan", "Masjid", "Labor AI"
43        );
```

```

44
45     positions_2005 = new HashMap<>();
46     positions_2005.put("Perpus", new Point(150, 80));
47     positions_2005.put("FTI", new Point(550, 80));
48     positions_2005.put("Rektorat", new Point(150, 160));
49     positions_2005.put("PKM", new Point(350, 160));
50     positions_2005.put("Lab Jaringan", new Point(550, 160));
51     positions_2005.put("Masjid", new Point(150, 240));
52     positions_2005.put("Labor AI", new Point(550, 240));
53
54     adjlist_2005 = new HashMap<>();
55     for (String node_2005 : vertices_2005) {
56         adjlist_2005.put(node_2005, new ArrayList<>());
57     }
58     addEdge_2005("Perpus", "FTI");
59     addEdge_2005("Perpus", "Rektorat");
60     addEdge_2005("Rektorat", "PKM");
61     addEdge_2005("PKM", "Lab Jaringan");
62     addEdge_2005("FTI", "Lab Jaringan");
63     addEdge_2005("Masjid", "Labor AI");
64     addEdge_2005("Lab Jaringan", "Labor AI");
65
66     currentPath_2005 = new ArrayList<>();
67     currentVisited_2005 = new HashSet<>();
68     currentExplored_2005 = 0;
69 }
70
71 private void addEdge_2005(String u_2005, String v_2005) {
72     adjlist_2005.get(u_2005).add(v_2005);
73     adjlist_2005.get(v_2005).add(u_2005);
74 }
75
76 private void setupGUI_2005() {
77     JPanel mainPanel_2005 = new JPanel();
78     mainPanel_2005.setLayout(new BorderLayout(mainPanel_2005, BorderLayout.Y_AXIS));
79     mainPanel_2005.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
80
81     JPanel controlPanel_2005 = new JPanel();
82     controlPanel_2005.setLayout(new FlowLayout(FlowLayout.LEFT, 20, 10));
83
84     JPanel startPanel_2005 = new JPanel(new FlowLayout(FlowLayout.LEFT, 5, 0));
85     startPanel_2005.add(new JLabel("Lokasi Awal :"));
86
87     comboStart_2005 = new JComboBox<>(new String[]{"Rektorat", "Perpus", "FTI", "PKM", "Lab Jaringan", "Masjid", "Labor AI"});
88     comboStart_2005.setSelectedItem("Rektorat");
89     startPanel_2005.add(comboStart_2005);
90
91     JPanel goalPanel_2005 = new JPanel(new FlowLayout(FlowLayout.LEFT, 5, 0));
92     goalPanel_2005.add(new JLabel("Lokasi Tujuan :"));
93     comboGoal_2005 = new JComboBox<>(new String[]{"Rektorat", "Perpus", "FTI", "PKM", "Lab Jaringan", "Masjid", "Labor AI"});
94     comboGoal_2005.setSelectedItem("Labor AI");
95     goalPanel_2005.add(comboGoal_2005);
96
97     controlPanel_2005.add(startPanel_2005);
98     controlPanel_2005.add(goalPanel_2005);
99
100     btnBFS_2005 = new JButton("BFS");
101     btnBFS_2005.setBackground(new Color(100, 200, 100));
102     btnBFS_2005.setForeground(color.BLACK);
103     btnBFS_2005.setFont(new Font("Arial", Font.BOLD, 12));
104
105     btnDFS_2005 = new JButton("DFS");
106     btnDFS_2005.setBackground(new Color(255, 165, 0));
107     btnDFS_2005.setForeground(color.BLACK);
108     btnDFS_2005.setFont(new Font("Arial", Font.BOLD, 12));
109
110     btnReset_2005 = new JButton("RESET");
111     btnReset_2005.setBackground(new Color(255, 100, 100));
112     btnReset_2005.setForeground(color.BLACK);
113     btnReset_2005.setFont(new Font("Arial", Font.BOLD, 12));
114
115     controlPanel_2005.add(btnBFS_2005);
116     controlPanel_2005.add(btnDFS_2005);
117     controlPanel_2005.add(btnReset_2005);
118
119     mainPanel_2005.add(controlPanel_2005);
120     mainPanel_2005.add(Box.createRigidArea(new Dimension(0, 10)));
121
122     JPanel graphContainer_2005 = new JPanel();
123     graphContainer_2005.setBorder(BorderFactory.createTitledBorder(
124         BorderFactory.createLineBorder(color.GRAY),
125         "VISUALISASI GRAF",
126         TitledBorder.LEFT,
127         TitledBorder.TOP,
128         new Font("Arial", Font.BOLD, 12)
129     ));

```

```

128    });
129    graphContainer_2005.setPreferredSize(new Dimension(860, 320));
130    graphContainer_2005.setLayout(new BorderLayout());
131
132    graphPanel_2005 = new GraphPanel_2005();
133    graphContainer_2005.add(graphPanel_2005, BorderLayout.CENTER);
134
135    mainPanel_2005.add(graphContainer_2005);
136    mainPanel_2005.add(Box.createRigidArea(new Dimension(0, 10)));
137
138    JPanel resultPanel_2005 = new JPanel();
139    resultPanel_2005.setLayout(new BoxLayout(resultPanel_2005, BoxLayout.Y_AXIS));
140    resultPanel_2005.setBorder(BorderFactory.createEmptyBorder(10, 0, 0, 0));
141
142    JLabel titleLabel_2005 = new JLabel("Hasil Pencarian :");
143    titleLabel_2005.setFont(new Font("Arial", Font.BOLD, 12));
144    resultPanel_2005.add(titleLabel_2005);
145    resultPanel_2005.add(Box.createRigidArea(new Dimension(0, 5)));
146
147    JLabel jalur_2005 = new JLabel("Jalur :");
148    resultPanel_2005.add(jalur_2005);
149    resultPanel_2005.add(Box.createRigidArea(new Dimension(0, 5)));
150
151    JLabel nodeDikunjungi_2005 = new JLabel("Node Dikunjungi :");
152    resultPanel_2005.add(nodeDikunjungi_2005);
153    resultPanel_2005.add(Box.createRigidArea(new Dimension(0, 5)));
154
155    JLabel jumlahNode_2005 = new JLabel("Jumlah Node Dikunjungi : 0");
156    resultPanel_2005.add(jumlahNode_2005);
157
158    mainPanel_2005.add(resultPanel_2005);
159
160    add(mainPanel_2005, BorderLayout.CENTER);
161
162    btnBFS_2005.addActionListener(e -> BFS());
163    btnDFS_2005.addActionListener(e -> DFS());
164    btnReset_2005.addActionListener(e -> resetGraph());
165    }
166
167    public void BFS() {
168        String start_2005 = (String) comboStart_2005.getSelectedItem();
169        String goal_2005 = (String) comboGoal_2005.getSelectedItem();
170
171        if (start_2005.equals(goal_2005)) {
172            JOptionPane.showMessageDialog(this, "Lokasi awal dan tujuan tidak boleh sama!", "Peringatan", JOptionPane.WARNING_MESSAGE);
173            return;
174        }
175
176        Queue<String> queue_2005 = new LinkedList<>();
177        Set<String> visited_2005 = new LinkedHashSet<>();
178        Map<String, String> parent_2005 = new HashMap<>();
179
180        queue_2005.add(start_2005);
181        visited_2005.add(start_2005);
182        int exploredCount_2005 = 0;
183        boolean found_2005 = false;
184
185        while (!queue_2005.isEmpty()) {
186            String current_2005 = queue_2005.poll();
187            exploredCount_2005++;
188
189            if (current_2005.equals(goal_2005)) {
190                found_2005 = true;
191                break;
192            }
193
194            for (String neighbor_2005 : adjList_2005.get(current_2005)) {
195                if (!visited_2005.contains(neighbor_2005)) {
196                    visited_2005.add(neighbor_2005);
197                    parent_2005.put(neighbor_2005, current_2005);
198                    queue_2005.add(neighbor_2005);
199                }
200            }
201        }
202
203        if (found_2005) {
204            List<String> path_2005 = new ArrayList<>();
205            String curr_2005 = goal_2005;
206            while (curr_2005 != null) {
207                path_2005.add(0, curr_2005);
208                curr_2005 = parent_2005.get(curr_2005);
209            }
210            currentPath_2005 = path_2005;
211            currentVisited_2005 = visited_2005;
212            currentExplored_2005 = exploredCount_2005;

```

```

213     displayPath();
214 } else {
215     labelJalur_2005.setText("Jalur : Tidak ditemukan!");
216     labelNodeDikunjungi_2005.setText("Node Dikunjungi :");
217     labelJumlahNode_2005.setText("Jumlah Node Dikunjungi : 0");
218 }
219 }
220
221 public void DFS() {
222     String start_2005 = (String) comboStart_2005.getSelectedItem();
223     String goal_2005 = (String) comboGoal_2005.getSelectedItem();
224
225     if (start_2005.equals(goal_2005)) {
226         JOptionPane.showMessageDialog(this, "Lokasi awal dan tujuan tidak boleh samal", "Peringatan", JOptionPane.WARNING_MESSAGE);
227         return;
228     }
229
230     Stack<String> stack_2005 = new Stack<>();
231     Set<String> visited_2005 = new LinkedHashSet<>();
232     Map<String, String> parent_2005 = new HashMap<>();
233
234     stack_2005.push(start_2005);
235     visited_2005.add(start_2005);
236     int exploredCount_2005 = 0;
237     boolean found_2005 = false;
238
239     while (!stack_2005.isEmpty()) {
240         String current_2005 = stack_2005.pop();
241         exploredCount_2005++;
242
243         if (current_2005.equals(goal_2005)) {
244             found_2005 = true;
245             break;
246         }
247
248         List<String> neighbors_2005 = new ArrayList<>(adjList_2005.get(current_2005));
249         Collections.reverse(neighbors_2005);
250
251         for (String neighbor_2005 : neighbors_2005) {
252             if (!visited_2005.contains(neighbor_2005)) {
253                 visited_2005.add(neighbor_2005);
254                 parent_2005.put(neighbor_2005, current_2005);
255                 stack_2005.push(neighbor_2005);
256             }
257         }
258     }
259
260     if (found_2005) {
261         List<String> path_2005 = new ArrayList<>();
262         String curr_2005 = goal_2005;
263         while (curr_2005 != null) {
264             path_2005.add(0, curr_2005);
265             curr_2005 = parent_2005.get(curr_2005);
266         }
267         currentPath_2005 = path_2005;
268         currentVisited_2005 = visited_2005;
269         currentExplored_2005 = exploredCount_2005;
270         displayPath();
271     } else {
272         labelJalur_2005.setText("Jalur : Tidak ditemukan!");
273         labelNodeDikunjungi_2005.setText("Node Dikunjungi :");
274         labelJumlahNode_2005.setText("Jumlah Node Dikunjungi : 0");
275     }
276 }
277
278 public void displayGraph() {
279     graphPanel_2005.repaint();
280 }
281
282 public void displayPath() {
283     StringBuilder pathStr_2005 = new StringBuilder();
284     for (int i_2005 = 0; i_2005 < currentPath_2005.size(); i_2005++) {
285         pathStr_2005.append(currentPath_2005.get(i_2005));
286         if (i_2005 < currentPath_2005.size() - 1) {
287             pathStr_2005.append(" -> ");
288         }
289     }
290
291     StringBuilder visitedStr_2005 = new StringBuilder();
292     int count_2005 = 0;
293     for (String node_2005 : currentVisited_2005) {
294         visitedStr_2005.append(node_2005);
295         if (count_2005 < currentVisited_2005.size() - 1) {
296             visitedStr_2005.append(", ");

```

```

297         }
298         count_2005++;
299     }
300 }
301
302 labelJalur_2005.setText("Jalur : " + pathStr_2005.toString());
303 labelNodeDikunjungi_2005.setText("Node Dikunjungi : " + visitedStr_2005.toString());
304 labelJumlahNode_2005.setText("Jumlah Node Dikunjungi : " + currentExplored_2005);
305
306 displayGraph();
307 }
308
309 public void resetGraph() {
310     currentPath_2005 = new ArrayList<>();
311     currentVisited_2005 = new HashSet<>();
312     currentExplored_2005 = 0;
313
314     labelJalur_2005.setText("Jalur :");
315     labelNodeDikunjungi_2005.setText("Node Dikunjungi :");
316     labelJumlahNode_2005.setText("Jumlah Node Dikunjungi : 0");
317
318     displayGraph();
319 }
320
321 class GraphPanel_2005 extends JPanel {
322     @Override
323     protected void paintComponent(Graphics g_2005) {
324         super.paintComponent(g_2005);
325         Graphics2D g2d_2005 = (Graphics2D) g_2005;
326         g2d_2005.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
327
328         Stroke dashed_2005 = new BasicStroke(2, BasicStroke.CAP_BUTT, BasicStroke.JOIN_BEVEL, 0, new float[]{9}, 0);
329         g2d_2005.setStroke(dashed_2005);
330         g2d_2005.setColor(color.GRAY);
331
332         for (String u_2005 : adjList_2005.keySet()) {
333             Point p1_2005 = positions_2005.get(u_2005);
334             for (String v_2005 : adjList_2005.get(u_2005)) {
335                 Point p2_2005 = positions_2005.get(v_2005);
336                 g2d_2005.drawLine(p1_2005.x, p1_2005.y, p2_2005.x, p2_2005.y);
337             }
338         }
339         if (currentPath_2005 != null && currentPath_2005.size() > 1) {
340             g2d_2005.setColor(color.RED);
341
342             g2d_2005.setStroke(new BasicStroke(3));
343             for (int i_2005 = 0; i_2005 < currentPath_2005.size() - 1; i_2005++) {
344                 Point p1_2005 = positions_2005.get(currentPath_2005.get(i_2005));
345                 Point p2_2005 = positions_2005.get(currentPath_2005.get(i_2005 + 1));
346                 g2d_2005.drawLine(p1_2005.x, p1_2005.y, p2_2005.x, p2_2005.y);
347             }
348             for (String node_2005 : vertices_2005) {
349                 Point p_2005 = positions_2005.get(node_2005);
350
351                 if (currentPath_2005 != null && currentPath_2005.contains(node_2005)) {
352                     g2d_2005.setColor(new Color(255, 200, 200));
353                 } else {
354                     g2d_2005.setColor(color.WHITE);
355                 }
356
357                 g2d_2005.fillOval(p_2005.x - 50, p_2005.y - 20, 100, 40);
358                 g2d_2005.setColor(color.BLACK);
359                 g2d_2005.setStroke(new BasicStroke(2));
360                 g2d_2005.drawOval(p_2005.x - 50, p_2005.y - 20, 100, 40);
361
362                 FontMetrics fm_2005 = g2d_2005.getFontMetrics();
363                 int width_2005 = fm_2005.stringWidth(node_2005);
364                 g2d_2005.drawString(node_2005, p_2005.x - width_2005 / 2, p_2005.y + 5);
365             }
366         }
367
368         public Dimension getPreferredSize() {
369             return new Dimension(840, 300);
370         }
371
372         public static void main(String[] args) {
373             SwingUtilities.invokeLater(() -> {
374                 PetaKampus_2511532005 app_2005 = new PetaKampus_2511532005();
375                 app_2005.setVisible(true);
376             });
377         }
378     }

```

Penjelasan kode dan fungsi :

- comboStart\_2005 = new JComboBox<>(new String[]{"Rektorat", "Perpus", ...});  
 comboGoal\_2005 = new JComboBox<>(new String[]{"Rektorat", "Perpus", ...});  
 berfungsi untuk memilih Lokasi awal dan tujuan.
- btnBFS\_2005.setBackground(new Color(100, 200, 100));  
 fungsi tombol dengan warna berbeda agar mudah dibedakan.

- c. `graphContainer_2005.setBorder(BorderFactory.createTitledBorder(..., "VISUALISASI GRAPH", ...));`  
panel dnegan border dan judul.
- d. `btnBFS_2005.addActionListener(e -> BFS());`  
`btnDFS_2005.addActionListener(e -> DFS());`  
`btnReset_2005.addActionListener(e -> resetGraph());`  
fungsi menghubungkan tombol dengan method yang akan dijalankan saat diklik.
- e. `public void BFS() {`  
`Queue<String> queue_2005 = new LinkedList<>();`  
`Set<String> visited_2005 = new LinkedHashSet<>();`  
`Map<String, String> parent_2005 = new HashMap<>();`  
Menemukan jalur terpendek.
- f. `public void DFS() {`  
`Stack<String> stack_2005 = new Stack<>(); // Tumpukan (LIFO)`  
`Set<String> visited_2005 = new LinkedHashSet<>();`  
`Map<String, String> parent_2005 = new HashMap<>();`  
Fungsi membalik urutan tetangga agar hasil DFS konsisten dan mirip dengan DFS rekursif.
- g. `public void displayPath() {`  
`labelJalur_2005.setText("Jalur : " + pathStr_2005.toString());`  
`labelNodeDikunjungi_2005.setText("Node Dikunjungi : " + visitedStr_2005.toString());`  
`labelJumlahNode_2005.setText("Jumlah Node Dikunjungi : " + currentExplored_2005);`  
`displayGraph(); // Refresh visualisasi`  
`}`  
Fungsi menampilkan hasil pencarian di 3 label dan memperbarui visualisasi graph.
- h. Inner class `GraphPanel_2005`  
Fungsi nya menggambar garid putus-putus, menggambar jalur yang ditemukan dengan garis merah tebal,.
- i. Method `resetGraph()`  
Fungsi menghapus semua hasil pencarian dan mengambilkan tampilan ke kondisi awal.
- j. Method `main()`  
Fugsinya menjalankan GUI di Event Dispatch Thread, dan membuat instance aplikasi dan menampilkan.

### 3. Output nya

